

Introducción a Docker y Kubernetes

UD 08. Caso práctico 01

- Aplicación Flask con Kubernetes



Autor: Sergi García Barea

Actualizado Enero 2026

Licencia



Reconocimiento – NoComercial - CompartirIgual (BY-NC-SA): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 **Importante**

 **Atención**

 **Interesante**

1. Introducción	2
2. Paso 0: iniciar MiniKube	2
3. Paso 1: Aplicación “app.py” y “Dockerfile” del caso práctico	3
4. Paso 2 (Comandos): Desplegando una aplicación mediante comandos	4
5. Paso 4: escalando nuestro despliegue	6
6. Paso 5 (Fichero YAML):Desplegando la aplicación mediante ficheros YAML	7
7. Paso 6: eliminando lo creado	9
8. Bibliografía	9

UD08. CASO PRÁCTICO 01

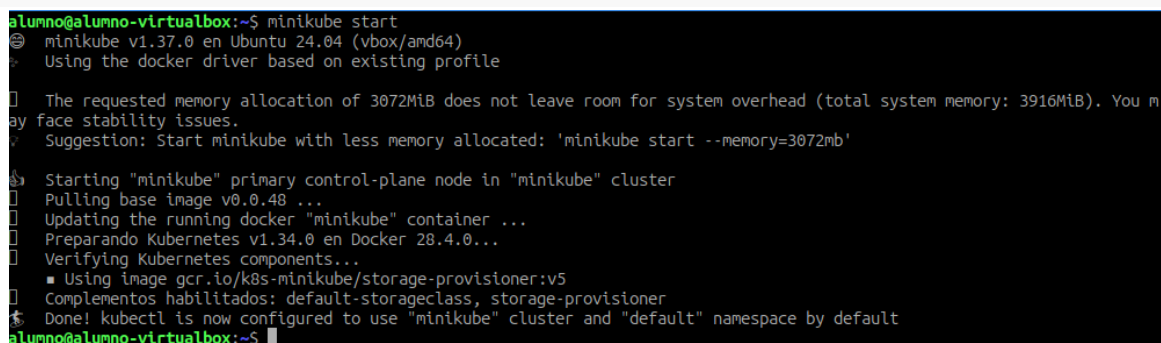
1. INTRODUCCIÓN

En este caso práctico vamos a poner en marcha una aplicación servidor con **“Python”** usando **“Flask”** <https://flask.palletsprojects.com/en/2.0.x/>. En el caso práctico desplegamos esta aplicación utilizando **“Kubernetes”** y **“MiniKube”**.

2. PASO 0: INICIAR MINIKUBE

Antes de empezar el caso práctico, debemos poner en marcha nuestro clúster con:

```
minikube start
```



```
alumno@alumno-virtualbox:~$ minikube start
minikube v1.37.0 en Ubuntu 24.04 (vbox/amd64)
Using the docker driver based on existing profile

The requested memory allocation of 3072MiB does not leave room for system overhead (total system memory: 3916MiB). You may face stability issues.
Suggestion: Start minikube with less memory allocated: 'minikube start --memory=3072mb'

Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.48 ...
Updating the running docker "minikube" container ...
Preparando Kubernetes v1.34.0 en Docker 20.4.0...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Complementos habilitados: default-storageclass, storage-provisioner
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
alumno@alumno-virtualbox:~$
```

Una vez puesto en marcha, podemos proseguir con el caso práctico.

3. PASO 1: APLICACIÓN “APP.PY” Y “DOCKERFILE” DEL CASO PRÁCTICO

Para el caso práctico, utilizaremos una sencilla aplicación web con **“Python”** y **“Flask”** que simplemente imprimirá el **“hostname”** (formado por identificador del contenedor) de quien está sirviendo la aplicación. El contenido de **“app.py”** es el siguiente:

```
from flask import Flask, escape, request
import socket
app = Flask(__name__)

@app.route('/')
def get_hostname():
    return "Aplicación servida desde hostname: "+socket.gethostname()
```

El contenido del fichero **“Dockerfile”** que incluimos comentado, es el siguiente:

```
#Utilizamos la imagen con Python 3.7
FROM python:3.7
#Copiamos del anfitrión a la imagen la aplicación
COPY app.py /app.py
#Instalamos la biblioteca Flask
RUN pip install flask
#Exponemos el puerto 5000
EXPOSE 5000
#Indicamos que se ejecute la aplicación al iniciar el contenedor
ENTRYPOINT env FLASK_APP=app.py flask run --host=0.0.0.0
```

Con esos dos ficheros, crearemos la imagen en nuestra máquina usando un comando similar a:

```
docker build -t cursodockerdocentes/flaskparakubernetes .
```

Tras lanzar esta orden, obtendremos algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ docker build -t cursodockerdocentes/flaskparakubernetes .
[+] Building 1.5s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile             0.1s
=> => transferring dockerfile: 379B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.7   1.0s
=> [internal] load .dockerignore                               0.0s
```

Una vez creada la imagen, podríamos crear un simple contenedor utilizando un comando similar a

#NO HACER, SOLO EJEMPLO

```
docker run -d --name miapp -p 5000:5000 cursodockerdocentes/flaskparakubernetes
```

pero en este caso, omitiremos este paso, ya que realizaremos el despliegue usando **“Kubernetes”**.

Una vez creada, deberemos subirla a un registro como **“Docker Hub”**. Podéis subirla a vuestra cuenta o **por comodidad simplemente usar una imagen subida a mi cuenta de “Docker Hub”**.

Enlace a la imagen: <https://hub.docker.com/r/cursodockerdocentes/flaskparakubernetes>

 **Importante:** se puede configurar para que la busque localmente, pero va “en contra” del propósito de **“Kubernetes”** (la idea es que podamos desplegar en un clúster distribuido, donde cada nodo pueda estar en una máquina).

4. PASO 2 (COMANDOS): DESPLEGANDO UNA APLICACIÓN MEDIANTE COMANDOS

Para desplegar esta aplicación en **“Kubernetes”** mediante comandos, en primer lugar crearemos un **“Pod”** en nuestro clúster mediante:

```
kubectl create deployment midespliegue --image=cursodockerdocentes/flaskparakubernetes
--port=5000
```

Obteniendo algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ kubectl create deployment midespliegue --image=cursodockerdocentes/flaskparakubernetes:latest --port=5000
deployment.apps/midespliegue created
```

Con esta orden estamos:

- Creando un **“Pod”** con nombre **“miapp”**.
- Indicando que el **“Pod”** utiliza la imagen **“cursodockerdocentes/flaskparakubernetes”**.
- Se expone el puerto 5000 de la aplicación.

Tras ello, podremos comprobar el estado de los **“Pod”** creado usando:

```
kubectl get pods
```

Obteniendo algo similar a (tardara igual unos minutos en estar en Running por la descarga):

```

alumno@alumno-virtualbox:~/Flask$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
midespliegue-5c6d9f4644-rcv4q      0/1     ContainerCreating   0           2m19s
alumno@alumno-virtualbox:~/Flask$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
midespliegue-5c6d9f4644-rcv4q      1/1     Running   0           4m9s

```

Una vez creado nuestro “**Pod**”, vamos a exponerlo como servicio para que se pueda acceder a nuestra aplicación. Podemos hacerlo con el siguiente comando:

```
kubectl expose deployment midespliegue --type=LoadBalancer --name=midespliegue-http
```

Obteniendo como respuesta:

```

alumno@alumno-virtualbox:~/Flask$ kubectl expose deployment midespliegue --type=LoadBalancer --name=midespliegue-http
service/midespliegue-http exposed

```

Una vez hecho esto, si examinamos los servicios:

```
kubectl get services
```

Obteniendo algo similar a:

```

alumno@alumno-virtualbox:~/Flask$ kubectl get services

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	6m20s
midespliegue-http	LoadBalancer	10.108.120.216	<pending>	5000:32295/TCP	28s

Si observamos detenidamente, la IP Externa de nuestro servicio está “**Pending**” y no lo tenemos expuesto directamente. Esto es debido a que todo el clúster “**Kubernetes**” está dentro de la máquina virtual de “**MiniKube**”. Para acceder al contenido, tenemos dos formas:

Forma 1: accederemos a la IP de “**MiniKube**” y nos expone el servicio en un puerto aleatorio.

```
minikube service midespliegue-http
```

A veces, el comando anterior da problemas. Si es así, podemos usar alternativamente, para exponer todos los servicios usando el parámetro “--all”.

```
minikube service --all
```

Tras lanzar este comando, se nos abrirá un navegador accediendo al servicio en uno de los puertos que expone “**MiniKube**” y aparecerá un texto similar a:

```

alumno@alumno-virtualbox:~/Flask$ minikube service midespliegue-http

```

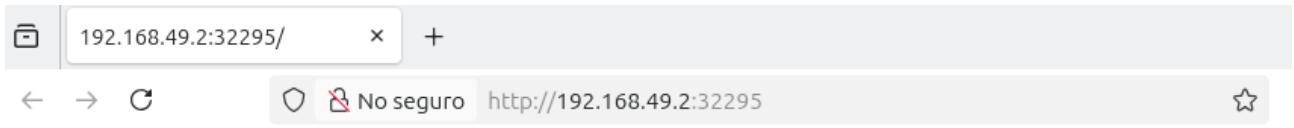
NAMESPACE	NAME	TARGET PORT	URL
default	midespliegue-http	5000	http://192.168.49.2:32295

```

[ ] Opening service default/midespliegue-http in default browser...
alumno@alumno-virtualbox:~/Flask$

```

Tras ello podemos observar que nuestra aplicación está siendo servida:



Aplicación servida desde hostname: midespliegue-5c6d9f4644-rcv4q

Forma 2: exponeremos el servicio con la IP de “**MiniKube**” y accederemos a él.

Para hacer esto, en una terminal aparte lanzaremos el siguiente comando

```
minikube tunnel
```

Este comando se ejecutará en la terminal “de forma indefinida” y mientras esté en funcionamiento, establecerá un túnel para acceder al servicio. Veremos algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ minikube tunnel
atus:
  machine: minikube
  pid: 50561
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [midespliegue-http]
errors:
  minikube: no errors
  router: no errors
  loadbalancer emulator: no errors
```

Si mientras este comando está en ejecución hacemos

```
kubectl get services
```

```
alumno@alumno-virtualbox: ~/Flask $ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	9m35s
midespliegue-http	LoadBalancer	10.108.120.216	10.108.120.216	5000:32295/TCP	3m43s

Ya observaremos una IP. En este ejemplo, accediendo al puerto 5000 de la IP del balanceador (en este ejemplo <http://10.108.174.125:5000>) accederemos a la aplicación Flask desplegada.

5. PASO 4: ESCALANDO NUESTRO DESPLIEGUE

Si queremos escalar el número de “**Pods**” de nuestro despliegue, podemos hacerlo de forma dinámica mediante comandos. Por ejemplo:

```
kubectl scale deployment midespliegue --replicas=3
```

Establecerá 3 réplicas. Si tras lanzarlo vemos los “**Pods**”:

```
kubectl get pods
```

Observamos algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ kubectl scale deployment midespliegue --replicas=3
deployment.apps/midespliegue scaled
alumno@alumno-virtualbox:~/Flask$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
midespliegue-5c6d9f4644-k6p7h      1/1     Running   0           19s
midespliegue-5c6d9f4644-np7m9      1/1     Running   0           19s
midespliegue-5c6d9f4644-rcv4q      1/1     Running   0           10m
alumno@alumno-virtualbox:~/Flask$
```

Donde se han creado 3 réplicas. Si accedemos a la máquina como se describió en el paso anterior, veremos que cada vez nos atenderá uno de los **“Pods”** gracias al balanceo de carga.

Si queremos que **“Kubernetes”** realice un autoescalado, nada tan fácil como ejecutar:

```
kubectl autoscale deployment midespliegue --min=5 --max=10
```

Observamos la siguiente imagen:

```
alumno@alumno-virtualbox:~/Flask$ kubectl autoscale deployment midespliegue --min=5 --max=10
horizontalpodautoscaler.autoscaling/midespliegue autoscaled
alumno@alumno-virtualbox:~/Flask$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
midespliegue-5c6d9f4644-6nf8q      1/1     Running   0           15s
midespliegue-5c6d9f4644-8m7f5      1/1     Running   0           15s
midespliegue-5c6d9f4644-k6p7h      1/1     Running   0           71s
midespliegue-5c6d9f4644-np7m9      1/1     Running   0           71s
midespliegue-5c6d9f4644-rcv4q      1/1     Running   0           11m
alumno@alumno-virtualbox:~/Flask$
```

Se observa que previo al autoescalado, había 3 **“Pods”**. A los pocos segundos (si no ocurre, podemos forzar recargando varias veces la petición al navegador), veremos que se ha aplicado el auto-escalado (con un mínimo de 5, pero la posibilidad de auto-escalarse a 10).

Si comprobamos el acceso como se explica en el **“Punto 3”**, veremos que nos sirve la aplicación hasta 5 contenedores distintos.

6. PASO 5 (FICHERO YAML): DESPLEGANDO LA APLICACIÓN MEDIANTE FICHEROS YAML

Antes de empezar, si hemos realizado el paso anterior, deberemos eliminar tanto el despliegue como el servicio donde hemos expuesto el mismo con los comandos:

```
kubectl delete deployment midespliegue
```

```
kubectl delete service midespliegue-http
```

También deberemos eliminar el autoescalado aplicado al despliegue con:

```
kubectl delete horizontalpodautoscaler midespliegue
```

Vamos a definir la configuración presentada al final del **“Paso 4”** utilizando un fichero **YAML** comentado **“deployment.yaml”** con el siguiente contenido.

```
#Indicamos La versión de La API
apiVersion: apps/v1
```

```
#Indicamos que este fichero es de un despliegue
kind: Deployment
#Metadatos del despliegue
metadata:
  name: midespliegue
#Características del despliegue
spec:
  # Al inicio 3 réplicas (luego si queremos activamos autoescalado)
  replicas: 3
  #Selector de los pods
  selector:
    matchLabels:
      app: midespliegue
#Plantilla de los pods
template:
  #Metadatos de los pods
  metadata:
    labels:
      app: midespliegue
  #Características de los pods
  spec:
    #Contenedor del pod
    containers:
      #Nombre, imagen y puerto expuesto
      - name: miapp
        image: cursodockerdocentes/flaskparakubernetes
        ports:
          - containerPort: 5000
```

Una vez listo, podemos lanzar nuestro despliegue usando el comando:

```
kubectl apply -f "deployment.yaml"
```

Con esto habremos creado nuestro despliegue. Observaremos algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ kubectl apply -f "deployment.yaml"
deployment.apps/midespliegue created
alumno@alumno-virtualbox:~/Flask$
```

Tras ello, deberemos crear nuestro servicio con un fichero “**service.yaml**”

```
#Versión de la API
apiVersion: v1
#Definimos un servicio
kind: Service
#Metadatos del servicio
metadata:
  name: midespliegue-http
#Características del servicio
spec:
  #Tipo de servicio
  type: LoadBalancer
  #Puerto a exponer
  ports:
    - port: 5000
      targetPort: 5000
  #A que aplica el servicio, busca los que coincidan con app: midespliegue para servir
  selector:
    app: midespliegue
```

Una vez listo, podemos lanzar nuestro despliegue usando el comando:


```
kubectl apply -f "service.yaml"
```

Si todo ha ido bien, tendremos algo similar a:

```
alumno@alumno-virtualbox:~/Flask$ kubectl apply -f "service.yaml"
service/midespliegue-http created
alumno@alumno-virtualbox:~/Flask$ kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes           ClusterIP   10.96.0.1      <none>          443/TCP          19m
midespliegue-http    LoadBalancer 10.101.211.43  10.101.211.43  5000:32525/TCP   40s
alumno@alumno-virtualbox:~/Flask$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
midespliegue  3/3     3            3           2m14s
alumno@alumno-virtualbox:~/Flask$
```

Si comprobamos el acceso como se explica en el “**Punto 3**”, veremos que nos sirve la aplicación correctamente.

Por último, definiremos mediante un fichero “**autoscale.yaml**” el auto-escalado del despliegue. El contenido del fichero será el siguiente:

```
apiVersion: autoscaling/v1
#Tipo autoescalado horizontal
kind: HorizontalPodAutoscaler
metadata:
  name: autoescalado
spec:
  #Indicamos a quien se aplica el auto-escalado
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: midespliegue
  #Mínimo y máximo de réplicas
  minReplicas: 5
  maxReplicas: 10
  #Máximo de CPU a usar durante el auto-escalado
  targetCPUUtilizationPercentage: 50
```

Una vez listo, podemos lanzar nuestro despliegue usando el comando:

```
kubectl apply -f "autoscale.yaml"
```

Para comprobar que el autoescalado se ha realizado correctamente, deberemos realizar una petición al servicio del despliegue, de forma similar a lo comentado en él “**Paso 3**”. Tras esto, escalará y veremos que el sistema ha autoescalado a 5 “**Pods**”.

```
alumno@alumno-virtualbox:~/Flask$ kubectl apply -f "autoscale.yaml"
horizontalpodautoscaler.autoscaling/autoescalado created
alumno@alumno-virtualbox:~/Flask$ minikube service midespliegue-http
minikube: no se encontró la orden
alumno@alumno-virtualbox:~/Flask$ minikube service midespliegue-http
```

NAMESPACE	NAME	TARGET PORT	URL
default	midespliegue-http	5000	http://192.168.49.2:32525

```
❏ Opening service default/midespliegue-http in default browser...
```

```
alumno@alumno-virtualbox:~/Flask$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
midespliegue-c54cbd949-2hgxz        1/1     Running   0           61s
midespliegue-c54cbd949-b9gqj        1/1     Running   0           5m25s
midespliegue-c54cbd949-kzhhz        1/1     Running   0           5m25s
midespliegue-c54cbd949-nkrq7        1/1     Running   0           61s
midespliegue-c54cbd949-rpc8n        1/1     Running   0           5m25s
alumno@alumno-virtualbox:~/Flask$
```

7. PASO 6: ELIMINANDO LO CREADO

Si queremos eliminar todos los elementos creados, podemos hacerlo con los siguientes comandos:

```
kubectl delete deployment midespliegue
kubectl delete service midespliegue-http
kubectl delete HorizontalPodAutoscaler autoescalado
```

8. BIBLIOGRAFÍA

- [1] Kubernetes <https://kubernetes.io/>
- [2] Kubernetes docs <https://kubernetes.io/docs/home/>
- [3] MiniKube <https://minikube.sigs.k8s.io/docs/>